Լաբորատոր աշխատանք 9
Դասի դինամիկ անդամներով խնդիրներ՝ մաթեմատիկական անալիզի թեմաներով

Աշխատանքի նպատակը

Ուսումնասիրել C++ ծրագրավորման լեզվում դինամիկ զանգվածների և դասերի կիրառումը, իրականացնել պարամետրերով և պատճենող կոնստրուկտորներ, վերագրման օպերատոր և դեստրուկտոր:Համասեռ դիֆերենցիալ հավասարման լուծման համար կիրառել $y = vx$ փոփոխականների փոխարինումը, Էյլերի թվային մեթոդի միջոցով գտնել v-ի և y-ի արժեքները, պահպանել դրանք առանձին դինամիկ զանգվածներում և հաշվել y-ի միջին թվաբանականը:

Խնդրի պահանջները

1. Յուրաքանչյուր խնդրի համար ստեղծել առանձին դաս՝ խնդրին համապատասխան անվանումով: Դասի `private` դաշտերում նախատեսել `double*` դինամիկ զանգվածներ և դրանց չափը:

2. Իրականացնել պարամետրերով կոնստրուկտոր, պատճենող կոնստրուկտոր, վերագրման օպերատոր և դեստրուկտոր (Rule of Three): Պատճենումը պետք է լինի խորքային:

3. Հաշվարկները կատարել առանձին մեթոդով, արդյունքները պահել դինամիկ զանգվածներում, ցուցադրել աղյուսակով և հաշվել խնդրում պահանջված ամփոփիչ ցուցանիշը:

4. Սովորական $y' = f(x, y)$ Կոշիի խնդիրների համար օգտագործել Էյլերի մեթոդը՝

$$y_{i+1} = y_i + h \cdot f(x_i, y_i)$$

Ընդունել $h = 0.05,\ N = 20$, եթե այլ բան նշված չէ:

5. Համասեռ հավասարումների դեպքում կիրառել $y = vx$ փոխարինումը՝

$$v' = \frac{f(1,v)-v}{x}$$

6. Ստուգել երկու անկախ օբյեկտ և դրանց խորքային պատճենումը: Մուտքային սխալ արժեքների դեպքում արտածել հասկանալի հաղորդագրություն:

Առաջադրանք

Համասեռ դիֆերենցիալ հավասարման լուծման համար կիրառել $y = vx$ փոփոխականների փոխարինումը, Էյլերի թվային մեթոդի միջոցով գտնել v-ի և y-ի արժեքները, պահպանել դրանք առանձին դինամիկ զանգվածներում և հաշվել y-ի միջին թվաբանականը:

```
#include <iostream>
#include <iomanip>
#include <cmath>
#include <stdexcept>
using namespace std;
```

```
class HomogeneousEquation {
private:
    double* x;
    double* v;
    double* y;
    int size;
    double h;
public:
    // Parametrized constructor
    HomogeneousEquation(int n, double step,
                        double x0, double y0) {
        if (n <= 0 || step <= 0 || x0 == 0) {
            throw invalid_argument("Invalid input values!");
        }
        size = n + 1;
        h = step;
        for (int i = 0; i < size; i++) {
            if (fabs(x0 + i * h) < 1e-12) {
                throw invalid_argument("x cannot be zero!");
            }
        }
        x = new double[size];
        v = new double[size];
        y = new double[size];
        x[0] = x0;
        y[0] = y0;
        v[0] = y0 / x0;
    }
    // Copy constructor
    HomogeneousEquation(const HomogeneousEquation& other) {
        size = other.size;
        h = other.h;
        x = new double[size];
        v = new double[size];
        y = new double[size];
        for (int i = 0; i < size; i++) {
            x[i] = other.x[i];
            v[i] = other.v[i];
            y[i] = other.y[i];
        }
    }
    // Assignment operator
    HomogeneousEquation& operator=(
        const HomogeneousEquation& other) {

        if (this != &other) {
            double* newX = new double[other.size];
            double* newV = new double[other.size];
            double* newY = new double[other.size];
            for (int i = 0; i < other.size; i++) {
                newX[i] = other.x[i];
```

```
            newV[i] = other.v[i];
            newY[i] = other.y[i];
        }
        delete[] x;
        delete[] v;
        delete[] y;
        x = newX;
        v = newV;
        y = newY;
        size = other.size;
        h = other.h;
    }
    return *this;
}
// Destructor
~HomogeneousEquation() {
    delete[] x;
    delete[] v;
    delete[] y;
}
// Calculation method (Euler method)
void calculate() {
    for (int i = 0; i < size - 1; i++) {

        x[i + 1] = x[i] + h;
        // v' = 1 / x
        v[i + 1] = v[i] + h / x[i];
        // y = v * x
        y[i + 1] = v[i + 1] * x[i + 1];
    }
}
// Display method
void display() const {
    cout << fixed << setprecision(4);
    cout << "\ni\t x\t\t v\t\t y\n";
    cout << "-----------------------------------------------\n";
    for (int i = 0; i < size; i++) {
        cout << i << "\t"
             << x[i] << "\t\t"
             << v[i] << "\t\t"
             << y[i] << endl;
    }
}
// Calculate average value of y
void findAverage() const {
    double sum = 0;
    for (int i = 0; i < size; i++) {
        sum += y[i];
    }
    double average = sum / size;
    cout << "\nAverage value of y = "
         << average << endl;
```

```
    }
};
int main() {
    int N;
    double h;
    cout << "Enter N: ";
    if (!(cin >> N) || N <= 0) {
        cout << "Invalid N!" << endl;
        return 1;
    }
    cout << "Enter h: ";
    if (!(cin >> h) || h <= 0) {
        cout << "Invalid h!" << endl;
        return 1;
    }
    try {
        // First object
        HomogeneousEquation first(N, h, 2.0, 1.0);
        first.calculate();
        first.display();
        first.findAverage();

        // Copy constructor
        HomogeneousEquation second(first);
        // Assignment operator
        HomogeneousEquation third(2, 0.1, 2.0, 1.0);
        third = first;
        cout << "\nCopy constructor and assignment operator "
             << "worked successfully!" << endl;
        second.findAverage();
        third.findAverage();
    }
    catch (const invalid_argument& e) {
        cout << "Error: " << e.what() << endl;
        return 1;
    }
    return 0;
}
```